

**Expt 2. Write a program to implement the data link layer framing method such as
1) character stuffing 2) bit stuffing**

Aim

To write and execute a program to implement Data Link Layer framing methods, namely:

1. Character Stuffing
2. Bit Stuffing

Theory

The Data Link Layer is responsible for reliable transmission of data between two directly connected devices. It divides the data into frames. To distinguish the beginning and end of each frame, special characters or bit patterns are used. If these special patterns appear in the actual data, stuffing techniques are employed to avoid ambiguity.

Two commonly used framing methods are:

- Character Stuffing
- Bit Stuffing

Bit Stuffing:

To avoid confusion with frame delimiter sequences (like 01111110 in HDLC), a 0 is inserted after every sequence of five consecutive 1s in the data stream.

Algorithm

1. Initialize a counter to 0.
2. Traverse each bit of the input data:
 - a. If the current bit is '1':
 - i. Append it to the result.
 - ii. Increment the counter.
 - iii. If counter == 5:
 - Append '0' (stuffing bit) to result.
 - Reset counter to 0.
 - b. Else (bit is '0'):
 - i. Append it to the result.
 - ii. Reset counter to 0.
3. Return the stuffed data.

Bit unstuffing:

1. Initialize a counter to 0.

2. Traverse each bit of the stuffed data:
 - a. If bit is '1':
 - i. Append it to result.
 - ii. Increment counter.
 - iii. If counter == 5:
 - Skip the next bit (it's the stuffed '0').
 - Reset counter to 0.
 - b. If bit is '0':
 - i. Append it.
 - ii. Reset counter to 0.
3. Return the unstuffed data.

Program:

```
def bit_stuffing(data):  
    stuffed_data = ""  
    count = 0  
    for bit in data:  
        if bit == '1':  
            count += 1  
            stuffed_data += bit  
            if count == 5:  
                stuffed_data += '0' # Stuffing '0' after five 1s  
                count = 0  
        else:  
            stuffed_data += bit  
            count = 0  
    return stuffed_data  
  
def bit_unstuffing(data):  
    unstuffed_data = ""  
    count = 0
```

```

i = 0
while i < len(data):
    bit = data[i]
    unstuffed_data += bit
    if bit == '1':
        count += 1
        if count == 5:
            i += 1 # Skip stuffed 0
            count = 0
    else:
        count = 0
    i += 1
return unstuffed_data

```

```

# Example

data = "11111011111101111"
stuffed = bit_stuffing(data)
unstuffed = bit_unstuffing(stuffed)

print("Original Data: ", data)
print("Stuffed Data: ", stuffed)
print("Unstuffed Data: ", unstuffed)

```

II. Character stuffing:

When control characters like FLAG or ESCAPE occur in the data, ESCAPE is inserted before them to distinguish actual data from control sequences.

For example:

- FLAG = F
- ESCAPE = E

Character stuffing: Algorithm

1. Initialize result with start flag character (e.g., 'F').
2. Traverse each character in the input:
 - a. If character is FLAG or ESC:

- i. Append ESC to result.
 - b. Append the character to result.
3. Append the FLAG at the end.
4. Return the result.

Character unstuffing:

1. Ignore the first and last characters (start and end FLAG).
2. Traverse the data:
 - a. If current character is ESC:
 - i. Move to the next character and append it to result.
 - b. Else:
 - i. Append current character to result.
3. Return the unstuffed data.

Program:

```
def char_stuffing(data, flag='F', esc='E'):
    stuffed_data = flag
    for char in data:
        if char == flag or char == esc:
            stuffed_data += esc # Escape the flag or escape character
        stuffed_data += char
    stuffed_data += flag
    return stuffed_data
```

```
def char_unstuffing(data, flag='F', esc='E'):
    unstuffed_data = ""
    i = 1 # Skip starting flag
    while i < len(data) - 1: # Skip ending flag
        if data[i] == esc:
            i += 1 # Skip escape and take next character
        unstuffed_data += data[i]
        i += 1
    return unstuffed_data
```

Example

```
data = "ABCFDEFG"
stuffed = char_stuffing(data)
unstuffed = char_unstuffing(stuffed)
```

```
print("\nOriginal Data: ", data)
print("Stuffed Data: ", stuffed)
print("Unstuffed Data: ", unstuffed)
```

Output:

Applications of Character Stuffing

- Character-oriented communication protocols.
- BISYNC (Binary Synchronous Communication).
- Text-based communication systems.
- Serial communication.

Applications of Bit Stuffing

- HDLC (High-Level Data Link Control)
- PPP (Point-to-Point Protocol)
- USB communication
- Wireless communication protocols

